

Exercise 1: Identify Synthesizability

The code given uses a for loop inside an always @(*) block to compute the sum of an array a. In general, synthesizable Verilog requires that all operations map to actual hardware elements. While some synthesis tools can unroll small loops at compile time, this code as written is not fully synthesizable, because the variable sum is updated in a combinational loop without proper initialization, and the number of iterations depends on n, which may not be a constant. To make it synthesizable, n must be a constant, and sum should be reset before accumulation.

Exercise 2: Fix the Code

The code attempts to create a clock by toggling clk using a delay (#10). The problem is that always begin without a sensitivity list and with # delays is not synthesizable, as delays are for simulation only. A synthesizable way to generate a clock is to rely on a real clock source from the FPGA board and not create it with # delays. For simulation, the corrected code can be:

```
always begin
```

```
    #10 clk = ~clk; // Toggle clock every 10 time units (simulation only)
```

```
end
```

But for synthesis, use the FPGA's clock pin directly.

Exercise 3: What hardware does this code describe?

```
always @(posedge clk) begin
```

```
    if (en)
```

```
        q <= d;
```

```
end
```

This code describes a D-type flip-flop with enable. On every rising edge of clk, if the enable signal en is high, the output q captures the value of d. If en is low, the output q retains its previous value. This is standard sequential logic used for registers or counters with an enable signal.